SASKEN TECHNOLOGIES

Technical MCQ Question Bank – Part 1

200 challenging MCQs with shuffled options and explanations

Purpose	Trainer-led first-round technical preparation and interview revision
Coverage	C, C++, DSA concepts, Operating Systems, DBMS, Computer Networks, Embedded Systems, Software Engineering
Level	Intermediate to hard; includes language rules, code behaviour, concepts and practical reasoning
Format	Four options per question, one best answer, followed by an explanation

How to use this bank: Ask students to attempt each section without looking at the explanation. Discuss why the correct option is right and why tempting distractors are wrong. Use the explanations to teach language rules and interview-level concepts.

Accuracy note: The shared Sasken recruitment email confirms a C or C++ programming assessment, technical topics, abstract reasoning and English communication, followed by technical and HR interviews. It does not publish an official MCQ paper or question distribution. These are original practice questions based on the role requirements and commonly relevant software-engineering interview topics—not leaked or verified previous-year questions.

Trainer edition • Part 1: MCQ bank only

Recommended teaching sequence

Teach in this order so that students build from language fundamentals to system-level interview concepts:

Section	Questions	Suggested focus
A. C programming	60	Pointers, arrays, memory, expressions, preprocessor, undefined behaviour
B. C++ programming	60	OOP, polymorphism, object lifetime, STL, templates, concurrency
C. Operating Systems	20	Processes, scheduling, synchronization, memory management
D. DBMS	15	Keys, normalization, transactions, SQL, indexing
E. Computer Networks	15	OSI/TCP-IP, TCP/UDP, DNS, DHCP, TLS
F. Embedded Systems	10	Microcontrollers, interrupts, firmware, I/O, DMA
G. Software Engineering	10	Testing, compiler/linker, debugging, API and version control
H. DSA concepts	10	Complexity, searching, sorting, graph traversal, hashing, trees

Trainer note: The next planned resources can cover pseudocode/output-tracing drills and a separate DSA coding workbook. This document deliberately focuses on the full MCQ bank first.

PART A — C PROGRAMMING

60 questions • Attempt first, then review the explanation.

Q1. In C, which statement about evaluating function arguments is correct?

A. Arguments are always evaluated left to right

B. Arguments are always evaluated right to left

C. Their evaluation order is generally unspecified

D. The compiler must evaluate them alphabetically

Answer: C — Their evaluation order is generally unspecified
Explanation: C does not generally specify the order in which function arguments are evaluated. Avoid depending on side effects across arguments.

Q2. What is the result category of reading an uninitialized automatic int variable?

A. It is guaranteed to be zero

B. Its value is indeterminate; reading it can cause undefined behaviour

C. It is always -1

D. It is implementation-defined

Answer: B — Its value is indeterminate; reading it can cause undefined behaviour
Explanation: An uninitialized automatic object has an indeterminate value. Evaluating it can lead to undefined behaviour.

Q3. Which declaration defines p as a pointer to a constant int?

A. const int *p

B. int const p*

C. int ** const p

D. int * const p

Answer: A — const int *p
Explanation: const int *p means the pointed-to int cannot be modified through p; p itself can be changed.

Q4. Which declaration defines a constant pointer to a mutable int?

A. const int const *p

B. int * const p

C. int const *p

D. const int *p

Answer: B — int * const p
Explanation: The const applies to the pointer object: p cannot be reassigned, but *p may be modified.

Q5. What does the expression sizeof(arr) / sizeof(arr[0]) produce when arr is an actual array in the same scope?

A. Always 1

B. Number of elements in arr

C. Number of bytes in arr

D. Size of the first element in bits

Answer: B — Number of elements in arr
Explanation: The total array size divided by one element's size gives the element count. This does not work after array-to-pointer decay.

Q6. What happens when an array parameter is declared as void f(int a[])?

A. The program is invalid

B. The parameter is adjusted to int *

C. The function receives the whole array by value

D. The parameter becomes int **

Answer: B — The parameter is adjusted to int *
Explanation: In a function parameter list, an array parameter is adjusted to a pointer parameter.

Q7. Which expression safely checks whether p is a null pointer?

A. sizeof(p) == 0

B. &p == 0

C. p == 0

D. *p == 0

Answer: C — p == 0
Explanation: A null pointer can be compared with 0 or NULL. Dereferencing p is not a valid null check.

Q8. Which is true about pointer arithmetic on int *p?

A. p + 1 advances by sizeof(p) bytes

B. p + 1 advances by sizeof(int) bytes

C. Pointer arithmetic is always byte-based

D. p + 1 advances by one byte

Answer: B — p + 1 advances by sizeof(int) bytes
Explanation: Pointer arithmetic advances in units of the pointed-to type.

Q9. What is the type of the string literal "Sasken" in C?

A. const char[6]

B. const char *

C. char[7]

D. char[6]

Answer: C — char[7]
Explanation: A string literal is an array of char including the terminating null character. Attempting to modify it has undefined behaviour.

Q10. What is the safest description of modifying a string literal in C?

A. It changes the literal for the whole program

B. Always allowed if stored in RAM

C. Allowed only through char *

D. Undefined behaviour

Answer: D — Undefined behaviour
Explanation: String literals are not modifiable; attempting to modify one has undefined behaviour.

Q11. Which function is intended to compare two null-terminated strings lexicographically?

A. strcpy

B. strlen

C. strcmp

D. strcat

Answer: C — strcmp
Explanation: strcmp compares strings and returns a value less than, equal to, or greater than zero.

Q12. What is the key risk of using gets()?

A. It cannot read spaces

B. It has no way to limit input length and can overflow the buffer

C. It only reads integers

D. It always appends two null characters

Answer: B — It has no way to limit input length and can overflow the buffer
Explanation: gets was removed from the C standard because it cannot prevent buffer overflow. Use fgets with a size limit.

Q13. Which allocation call initializes allocated bytes to all-bits-zero?

A. aligned_alloc

B. malloc

C. calloc

D. realloc

Answer: C — calloc
Explanation: calloc allocates an array and initializes all bytes to zero.

Q14. If realloc(p, new_size) fails for a nonzero size, what happens to the original allocation?

A. It is partially resized

B. The program must terminate

C. It is automatically freed

D. It remains valid and unchanged

Answer: D — It remains valid and unchanged
Explanation: On failure realloc returns NULL and the original block remains allocated. Assigning directly to p can lose the original pointer.

Q15. Which is the correct safe pattern for realloc?

A. realloc always succeeds

B. free(p); p = realloc(p, n)

C. p = realloc(p, n)

D. tmp = realloc(p, n); if (tmp) p = tmp

Answer: D — tmp = realloc(p, n); if (tmp) p = tmp
Explanation: Use a temporary pointer so the original allocation is not lost if realloc fails.

Q16. What does free(NULL) do?

A. Has no effect

B. Frees the entire heap

C. Causes undefined behaviour

D. Allocates zero bytes

Answer: A — Has no effect
Explanation: The C standard specifies that free(NULL) has no effect.

Q17. Which statement about a pointer after free(p) is correct?

A. It becomes a dangling pointer if its value is retained

B. It can safely be dereferenced once

C. It becomes NULL automatically

D. It points to the next allocation

Answer: A — It becomes a dangling pointer if its value is retained
Explanation: free does not change the pointer variable. Its value becomes indeterminate for use as a pointer to the released object; assigning NULL is a common defensive practice.

Q18. What is the lifetime of a block-scope variable declared static?

A. Only while the block executes

B. Until the next loop iteration

C. Until the function returns

D. The entire program execution

Answer: D — The entire program execution
Explanation: A static local variable has static storage duration and retains its stored value between function calls.

Q19. What is the linkage of a file-scope function declared static?

A. No linkage

B. Dynamic linkage

C. External linkage

D. Internal linkage

Answer: D — Internal linkage
Explanation: A file-scope static function is visible only within that translation unit.

Q20. Which storage-class specifier is used to declare an object defined in another translation unit?

A. typedef

B. extern

C. register

D. auto

Answer: B — extern
Explanation: extern declares an identifier with external linkage in common use, referring to a definition elsewhere.

Q21. What is the result of applying sizeof to a function type?

A. Constraint violation; sizeof cannot be applied to a function type

B. Always 1

C. Size of return type

D. Size of function code

Answer: A — Constraint violation; sizeof cannot be applied to a function type
Explanation: sizeof is not permitted on function types or incomplete types (with specified exceptions for VLA operands).

Q22. Which operator has higher precedence?

A. Equality ==

B. Bitwise &

C. Logical &&

D. Assignment =

Answer: A — Equality ==
Explanation: Equality operators bind more tightly than bitwise AND, which binds more tightly than logical AND; assignment is lower.

Q23. What is the value of (5 & 3) in C?

A. 1

B. 8

C. 2

D. 7

Answer: A — 1
Explanation: Binary 0101 AND 0011 gives 0001, which is 1.

Q24. What is the value of (5 ^ 3) in C?

A. 6

B. 1

C. 8

D. 7

Answer: A — 6
Explanation: Binary 0101 XOR 0011 gives 0110, which is 6.

Q25. For an unsigned integer x, what does x >> 1 generally do?

A. Rotates bits right

B. Shifts bits right, filling high bits with zero

C. Performs signed division in all cases

D. Multiplies x by two modulo range

Answer: B — Shifts bits right, filling high bits with zero
Explanation: Right shift of an unsigned value is a logical shift with zero fill.

Q26. Which shift operation can have undefined behaviour in C?

A. Shifting unsigned right by one

B. Shifting unsigned by zero

C. Shifting a value by a negative count

D. Shifting 8u left by one when representable

Answer: C — Shifting a value by a negative count
Explanation: A negative shift count or count at least the width of the promoted left operand is undefined. Signed left shift has additional constraints.

Q27. What is the result of integer division 7 / 2 in C when both operands are int?

A. Undefined behaviour

B. 3.5

C. 4

D. 3

Answer: D — 3
Explanation: Integer division discards the fractional part, truncating toward zero.

Q28. What is the result of -7 / 2 in modern C?

A. -3.5

B. -3

C. -4

D. Undefined behaviour

Answer: B — -3
Explanation: Integer division truncates toward zero, so -7/2 is -3.

Q29. What is the value of the expression sizeof('A') in C?

A. Implementation-defined as pointer size

B. sizeof(char)

C. sizeof(int)

D. Always 1

Answer: C — sizeof(int)
Explanation: Unlike C++, a character constant such as 'A' has type int in C.

Q30. Which statement about enum constants in C is correct?

A. They are string objects

B. They are pointers

C. They have integer constant values

D. They are always unsigned long

Answer: C — They have integer constant values
Explanation: Enumeration constants are integer constant expressions.

Q31. Which is true about a union?

A. Its size is the sum of all member sizes

B. Its members share storage

C. It cannot contain a structure

D. All members have separate storage

Answer: B — Its members share storage
Explanation: A union overlays its members in the same storage region; its size is sufficient for its largest member, including alignment.

Q32. What does a flexible array member look like?

A. int [] a; anywhere in a struct

B. int a[]; as the final member of a struct

C. A zero-length array in every compiler

D. int *a[];

Answer: B — int a[]; as the final member of a struct
Explanation: A flexible array member is an incomplete array type as the last member of a structure, with special allocation considerations.

Q33. What does restrict communicate to the compiler?

A. The pointer is constant

B. The object is read-only

C. Based on the restrict association rules, accesses use that pointer as the designated means

D. The pointer cannot be null

Answer: C — Based on the restrict association rules, accesses use that pointer as the designated means
Explanation: restrict is an optimization promise about access to an object during a block execution; violating its rules can cause undefined behaviour.

Q34. Which statement about volatile is correct?

A. It tells the implementation that accesses are observable and should not be elided as ordinary accesses

B. It makes operations thread-safe

C. It flushes CPU cache in all systems

D. It guarantees atomic access

Answer: A — It tells the implementation that accesses are observable and should not be elided as ordinary accesses
Explanation: volatile is not a synchronization primitive and does not guarantee atomicity or thread safety.

Q35. Which header declares printf?

A. <stdlib.h>

B. <math.h>

C. <string.h>

D. <stdio.h>

Answer: D — <stdio.h>
Explanation: The standard input/output functions are declared in stdio.h.

Q36. Which header declares malloc and free?

A. <stddef.h>

B. <string.h>

C. <stdlib.h>

D. <stdio.h>

Answer: C — <stdlib.h>
Explanation: The allocation functions are declared in stdlib.h.

Q37. What is the purpose of a function prototype?

A. It executes the function

B. It allocates the function's stack frame

C. It defines global variables

D. It provides the function type information to the compiler

Answer: D — It provides the function type information to the compiler
Explanation: A prototype allows compile-time checking of calls and conversions against the declared parameter and return types.

Q38. What is the difference between #define MAX 10 and const int max = 10 in C?

A. A macro allocates no tokens

B. They are identical in every respect

C. The macro is preprocessor text substitution; const defines a typed object

D. const is always usable in every integer constant expression

Answer: C — The macro is preprocessor text substitution; const defines a typed object
Explanation: Macros are processed before compilation and do not have C object type; const objects are typed objects.

Q39. What does the # operator do inside a function-like macro replacement list?

A. Token pasting

B. Stringification of a macro argument

C. Comment removal

D. Conditional compilation

Answer: B — Stringification of a macro argument
Explanation: The stringification operator converts the corresponding macro argument tokens into a string literal.

Q40. What does ## do in a macro replacement list?

A. Stringifies an argument

B. Ends macro expansion

C. Starts a comment

D. Concatenates preprocessing tokens

Answer: D — Concatenates preprocessing tokens
Explanation: The token-pasting operator combines adjacent preprocessing tokens.

Q41. Which statement about the comma operator is correct?

A. It sequences the left operand before the right and yields the right operand's value

B. It is lower precedence than assignment

C. It evaluates only the right operand

D. It has the same meaning as comma in function arguments

Answer: A — It sequences the left operand before the right and yields the right operand's value
Explanation: The comma operator sequences its left expression before its right expression and has the right expression's value.

Q42. What is the result of the conditional operator a ? b : c?

A. It always evaluates c first

B. Both b and c are evaluated

C. It is equivalent to bitwise OR

D. Only the selected second or third operand is evaluated

Answer: D — Only the selected second or third operand is evaluated
Explanation: The condition is evaluated first; only one of the second and third operands is evaluated.

Q43. What is the role of a null character in a C string?

A. Marks the start of the string

B. Marks the end of the string

C. Stores the string length

D. Separates words

Answer: B — Marks the end of the string
Explanation: String-handling functions conventionally scan until the terminating null character.

Q44. What is the behaviour of strcpy when destination is too small?

A. It writes beyond the destination, causing undefined behaviour

B. It reallocates destination

C. It truncates safely

D. It reports an error code

Answer: A — It writes beyond the destination, causing undefined behaviour
Explanation: strcpy has no destination-size argument. The caller must ensure sufficient space.

Q45. Which function reads at most n-1 characters and appends a null terminator when successful?

A. getc

B. fgets

C. puts

D. scanf("%s")

Answer: B — fgets
Explanation: fgets reads at most size-1 characters and null-terminates the resulting string.

Q46. What is the purpose of EOF?

A. A character stored in every file

B. A file descriptor

C. A null terminator

D. A negative int macro indicating end-of-file or read error

Answer: D — A negative int macro indicating end-of-file or read error
Explanation: EOF is an int-valued macro distinct from every unsigned-char value; check ferror/feof to distinguish errors and end-of-file.

Q47. Which statement about signed integer overflow is correct in C?

A. It always raises an exception

B. It saturates at INT_MAX

C. It is undefined behaviour

D. It wraps modulo 2^n

Answer: C — It is undefined behaviour
Explanation: Signed integer overflow is undefined behaviour in C.

Q48. Which statement about unsigned integer overflow is correct?

A. Saturates at maximum

B. Raises SIGFPE

C. Undefined behaviour

D. Wraps modulo one more than the maximum value

Answer: D — Wraps modulo one more than the maximum value
Explanation: Unsigned arithmetic is defined modulo 2^N for the type's value width.

Q49. What does the expression *p++ mean?

A. Increment p after the dereference expression is formed

B. Invalid syntax

C. Increment the pointer target twice

D. Increment the pointed-to object

Answer: A — Increment p after the dereference expression is formed
Explanation: Postfix ++ binds more tightly than unary *. It is equivalent to *(p++).

Q50. What does the expression (*p)++ mean?

A. Move p to next object

B. Dereference after decrement

C. Increment the pointer

D. Increment the pointed-to object

Answer: D — Increment the pointed-to object
Explanation: Parentheses make the postfix increment apply to the object pointed to by p.

Q51. What is a translation unit?

A. A source file after preprocessing, with included content

B. A single function

C. A single statement

D. An executable process

Answer: A — A source file after preprocessing, with included content
Explanation: A translation unit is the source file after preprocessing, including included headers and expanded macros.

Q52. What is the purpose of a header guard?

A. To allocate static memory

B. To speed up execution

C. To export all functions

D. To prevent repeated inclusion of the same header contents

Answer: D — To prevent repeated inclusion of the same header contents
Explanation: A conventional #ifndef/#define/#endif guard prevents duplicate processing within a translation unit.

Q53. Which statement about recursion and stack usage is most accurate?

A. Recursion never uses stack space

B. Recursion always runs in O(1) space

C. Each active call generally requires call-state storage, often on a stack

D. The compiler must convert every recursion to a loop

Answer: C — Each active call generally requires call-state storage, often on a stack
Explanation: Recursive calls usually consume call-stack space; exact implementation details and optimization are compiler-dependent.

Q54. What is the effective type of an object with allocated storage and no declared type?

A. Always char

B. It can be established by a non-character lvalue access or by copying with memcpy, subject to the standard's rules

C. It is always int

D. Always void

Answer: B — It can be established by a non-character lvalue access or by copying with memcpy, subject to the standard's rules
Explanation: Allocated storage has no declared type; effective type is determined by the rules for accesses and copies.

Q55. What does _Alignof (or alignof in C11 via stdalign.h) provide?

A. The number of members in a structure

B. The size of an object

C. The address of a variable

D. The alignment requirement of a type

Answer: D — The alignment requirement of a type
Explanation: Alignment queries report the implementation's alignment requirement for a type.

Q56. What is a compound literal in C?

A. An unnamed object initialized using a type name followed by an initializer list

B. A string macro

C. A nested function

D. A pointer cast

Answer: A — An unnamed object initialized using a type name followed by an initializer list
Explanation: Compound literals create unnamed objects with block or static storage duration depending on context.

Q57. Which statement about designated initializers in C is correct?

A. They initialize only pointers

B. They are available only in C++

C. They allow selected members or array elements to be initialized by designator

D. They require runtime input

Answer: C — They allow selected members or array elements to be initialized by designator
Explanation: Designated initializers such as .member = value make aggregate initialization explicit.

Q58. What is the purpose of _Generic in C11?

A. Dynamic memory allocation

B. Thread creation

C. Compile-time selection of an expression based on the type of a controlling expression

D. Runtime reflection

Answer: C — Compile-time selection of an expression based on the type of a controlling expression
Explanation: _Generic supports type-generic selection at translation time.

Q59. Which statement about setjmp and longjmp is correct?

A. They provide non-local jumps, with restrictions on modified non-volatile automatic variables

B. They allocate heap memory

C. They are used for C++ exception handling

D. They are ordinary loop statements

Answer: A — They provide non-local jumps, with restrictions on modified non-volatile automatic variables
Explanation: setjmp/longjmp can transfer control non-locally; certain automatic variables changed after setjmp may become indeterminate.

Q60. What does the restrict qualifier primarily help express?

A. The object is immutable

B. A promise about restricted aliasing/access paths that can enable optimization

C. The pointer is shared between threads

D. The pointer is always valid

Answer: B — A promise about restricted aliasing/access paths that can enable optimization
Explanation: restrict is an aliasing contract, not a nullability, immutability or thread-safety qualifier.

PART B — C++ PROGRAMMING

60 questions • Attempt first, then review the explanation.

Q61. Which declaration makes a member function callable on a const object?

A. const void f()

B. static void f() const

C. void f() const

D. void f()

Answer: C — void f() const
Explanation: A trailing const qualifies the implicit object parameter and promises not to modify the object through this.

Q62. What is the default access level for members of a class?

A. protected

B. package

C. public

D. private

Answer: D — private
Explanation: Members of a class are private by default; struct members are public by default.

Q63. What is the default access level for members of a struct in C++?

A. public

B. protected

C. private

D. internal

Answer: A — public
Explanation: C++ struct members and base inheritance are public by default.

Q64. Which member function cannot be virtual?

A. Pure virtual function

B. Constructor

C. Ordinary member function

D. Destructor

Answer: B — Constructor
Explanation: Constructors cannot be virtual because the object is not yet fully constructed for virtual dispatch.

Q65. Why should a polymorphic base class often have a virtual destructor?

A. To enable function overloading

B. To reduce object size

C. To make constructors virtual

D. To ensure derived destruction when deleting through a base pointer

Answer: D — To ensure derived destruction when deleting through a base pointer
Explanation: Deleting a derived object through a base pointer requires a virtual base destructor for correct derived cleanup.

Q66. What happens if a derived class hides a base overload set by declaring a same-named function?

A. The program always fails to compile

B. The base overloads can be hidden by name lookup

C. All base overloads remain automatically visible

D. The derived function becomes static

Answer: B — The base overloads can be hidden by name lookup
Explanation: Name hiding applies; a using declaration can bring base overloads into the derived scope.

Q67. What is object slicing?

A. Removing virtual functions

B. Deleting a derived class

C. Splitting a vector

D. Copying only the base subobject when a derived object is copied by value into a base object

Answer: D — Copying only the base subobject when a derived object is copied by value into a base object
Explanation: Passing or assigning a derived object by value as a base object discards derived-specific state.

Q68. Which cast is intended for checked downcasting in a polymorphic hierarchy?

A. dynamic_cast

B. static_cast

C. const_cast

D. reinterpret_cast

Answer: A — dynamic_cast
Explanation: dynamic_cast performs a runtime-checked downcast for polymorphic types; pointer casts can yield nullptr on failure.

Q69. What does static_cast generally do?

A. Reinterprets arbitrary object representation

B. Removes const only

C. Performs runtime type checking for all casts

D. Performs compile-time-checked conversions defined by the language

Answer: D — Performs compile-time-checked conversions defined by the language
Explanation: static_cast handles specified conversions but does not provide general runtime type checking.

Q70. What does const_cast change?

A. The inheritance graph

B. The dynamic type

C. The const/volatile qualification

D. The object size

Answer: C — The const/volatile qualification
Explanation: const_cast adds or removes cv-qualification; modifying an object originally defined const through a cast is undefined behaviour.

Q71. What is the purpose of reinterpret_cast?

A. Copy construction

B. Low-level reinterpretation between certain pointer/reference/integer types

C. Safe polymorphic downcast

D. Exception handling

Answer: B — Low-level reinterpretation between certain pointer/reference/integer types
Explanation: reinterpret_cast performs low-level conversions and does not make an otherwise invalid object access safe.

Q72. What is a pure virtual function declared as?

A. virtual void f();

B. virtual void f() = 1;

C. void f() = 0;

D. abstract void f();

Answer: C — void f() = 0;
Explanation: A pure virtual function uses = 0 in its declaration.

Q73. Can an abstract class have a constructor?

A. Only if all methods are static

B. Yes

C. Only in C++20

D. No

Answer: B — Yes
Explanation: Abstract classes can have constructors; derived constructors invoke them to initialize the base subobject.

Q74. What is the rule for a virtual function call from a base constructor?

A. It is always a compile error

B. It calls every override

C. It dispatches to the most-derived override

D. It dispatches only within the class currently being constructed

Answer: D — It dispatches only within the class currently being constructed
Explanation: During base construction/destruction, virtual dispatch does not reach more-derived portions not yet constructed or already destroyed.

Q75. What is a covariant return type in virtual overriding?

A. A return type with const only

B. A return type that is any unrelated type

C. A template return type

D. A derived pointer/reference return type compatible with the base pointer/reference return type

Answer: D — A derived pointer/reference return type compatible with the base pointer/reference return type
Explanation: Covariance permits an overriding function to return a pointer/reference to a more-derived class under specified conditions.

Q76. What is the Rule of Three traditionally concerned with?

A. Destructor, copy constructor and copy assignment when managing resources

B. Three constructors

C. Three template parameters

D. Three inheritance levels

Answer: A — Destructor, copy constructor and copy assignment when managing resources
Explanation: If a class manages a resource and needs one of these special members, it may need all three.

Q77. What does the Rule of Five add?

A. Two virtual functions

B. Two friend functions

C. Default constructor and conversion operator

D. Move constructor and move assignment operator

Answer: D — Move constructor and move assignment operator
Explanation: The Rule of Five adds move construction and move assignment to the Rule of Three.

Q78. What does the Rule of Zero recommend?

A. Never use classes

B. Always define five special members

C. Prefer composing types that manage resources so the class needs no custom special member functions

D. Use only raw pointers

Answer: C — Prefer composing types that manage resources so the class needs no custom special member functions
Explanation: Resource-managing standard library types allow classes to avoid custom resource-management special members.

Q79. What is RAII?

A. Resource Acquisition Is Initialization

B. Runtime Array Initialization Interface

C. Register Allocation in Instructions

D. Random Access Inheritance Implementation

Answer: A — Resource Acquisition Is Initialization
Explanation: RAII binds resource acquisition and release to object lifetime, typically constructors and destructors.

Q80. What does std::unique_ptr model?

A. Shared ownership

B. Non-owning reference

C. A raw array only

D. Exclusive ownership

Answer: D — Exclusive ownership
Explanation: unique_ptr is a move-only smart pointer expressing exclusive ownership by default.

Q81. What does std::shared_ptr use to manage shared ownership?

A. A global garbage collector

B. A compile-time singleton

C. The stack pointer

D. A reference-counted control block

Answer: D — A reference-counted control block
Explanation: shared_ptr instances share ownership via a control block, typically with reference counting.

Q82. What is a common cause of shared_ptr cycles?

A. unique_ptr use

B. Stack allocation

C. Weak references

D. Two or more shared_ptr objects owning each other

Answer: D — Two or more shared_ptr objects owning each other
Explanation: Cycles keep reference counts nonzero; weak_ptr can break ownership cycles.

Q83. What is std::weak_ptr used for?

A. Owning an object exclusively

B. Observing an object managed by shared_ptr without extending its lifetime

C. Replacing all references

D. Allocating raw memory

Answer: B — Observing an object managed by shared_ptr without extending its lifetime
Explanation: weak_ptr is a non-owning observer of shared ownership and can be promoted temporarily with lock().

Q84. Which operation is generally invalidated by vector reallocation?

A. No iterators

B. All iterators, references and pointers to its elements

C. Only the vector object's address

D. Only iterators to the first element

Answer: B — All iterators, references and pointers to its elements
Explanation: Reallocation moves elements to new storage, invalidating iterators, references and pointers to elements.

Q85. What is the average complexity of std::unordered_map lookup?

A. O(n log n)

B. O(log n)

C. O(n^2)

D. O(1)

Answer: D — O(1)
Explanation: Average lookup is constant time, though worst-case lookup can be linear.

Q86. What is the typical complexity of std::map lookup?

A. O(1)

B. O(2^n)

C. O(log n)

D. O(n^2)

Answer: C — O(log n)
Explanation: std::map is typically an ordered balanced tree with logarithmic lookup.

Q87. Which container provides contiguous storage for its elements?

A. std::map

B. std::list

C. std::vector

D. std::set

Answer: C — std::vector
Explanation: vector stores elements contiguously, except bool specialization details.

Q88. Which container supports efficient insertion/removal at both ends?

A. std::map

B. std::deque

C. std::set

D. std::array

Answer: B — std::deque
Explanation: deque supports efficient insertion/removal at both ends and random access.

Q89. What is the difference between emplace_back and push_back?

A. emplace_back always copies

B. push_back is only for integers

C. emplace_back constructs an element in place from arguments

D. They are unrelated to containers

Answer: C — emplace_back constructs an element in place from arguments
Explanation: emplace_back forwards constructor arguments to construct the element in place; push_back inserts an existing value.

Q90. What is perfect forwarding typically implemented with?

A. const_cast

B. std::forward and forwarding references

C. reinterpret_cast

D. std::move only

Answer: B — std::forward and forwarding references
Explanation: std::forward preserves the value category of a deduced forwarding-reference argument.

Q91. What does std::move do?

A. Casts an expression to an rvalue reference-like value category

B. Moves an object immediately

C. Calls the move constructor unconditionally

D. Copies and deletes the source

Answer: A — Casts an expression to an rvalue reference-like value category
Explanation: std::move is a cast; a move operation occurs only if an appropriate overload is selected.

Q92. What is an rvalue reference declared with?

A. T&&

B. const T&

C. T*

D. T&

Answer: A — T&&
Explanation: T&& is an rvalue reference, with special template deduction rules for forwarding references.

Q93. What is a lambda capture by reference?

A. []

B. [&]

C. [=]

D. [this] only

Answer: B — [&]
Explanation: [&] captures used local variables by reference by default.

Q94. What does [=] generally mean in a lambda capture list?

A. Capture all variables by reference

B. Capture used automatic local variables by value

C. Capture only this

D. Capture no variables

Answer: B — Capture used automatic local variables by value
Explanation: [=] is the default by-copy capture for eligible odr-used local variables.

Q95. What is a generic lambda?

A. A lambda using auto in its parameter list

B. A lambda with no body

C. A lambda that captures globals only

D. A lambda with virtual methods

Answer: A — A lambda using auto in its parameter list
Explanation: Auto parameters make the lambda's call operator a template.

Q96. What is SFINAE?

A. Safe Function Invocation And Evaluation

B. Static Function Inheritance And Extension

C. Substitution Failure Is Not An Error

D. Single File Interface And Execution

Answer: C — Substitution Failure Is Not An Error
Explanation: In certain template substitution contexts, an invalid substitution removes a candidate rather than making the program immediately ill-formed.

Q97. What is a variadic template?

A. A template accepting zero or more template arguments

B. A template with exactly two types

C. A runtime-only template

D. A template without parameters

Answer: A — A template accepting zero or more template arguments
Explanation: Parameter packs allow templates to accept a variable number of template arguments.

Q98. What is the purpose of std::optional?

A. Guarantee heap allocation

B. Store only exceptions

C. Share ownership

D. Represent a value that may be absent

Answer: D — Represent a value that may be absent
Explanation: optional represents an optional contained value without requiring a separate heap allocation.

Q99. What does noexcept specify?

A. The function is asynchronous

B. The function cannot return

C. The function is declared not to throw exceptions under its contract

D. The function is virtual

Answer: C — The function is declared not to throw exceptions under its contract
Explanation: noexcept is an exception specification; violating a non-throwing exception specification leads to termination.

Q100. What happens when an exception escapes a noexcept function?

A. The function is retried

B. std::terminate is called

C. It is silently ignored

D. The caller catches it automatically

Answer: B — std::terminate is called
Explanation: An exception escaping a noexcept function causes std::terminate.

Q101. What is the purpose of an explicit constructor?

A. Make the constructor virtual

B. Make the class abstract

C. Prevent certain unintended implicit conversions

D. Enable implicit conversions

Answer: C — Prevent certain unintended implicit conversions
Explanation: explicit prevents implicit conversion sequences that would otherwise use the constructor.

Q102. What is a delegating constructor?

A. A constructor inherited from a friend

B. A virtual constructor

C. A constructor that returns a pointer

D. A constructor that calls another constructor of the same class

Answer: D — A constructor that calls another constructor of the same class
Explanation: A delegating constructor uses another constructor in the same class's member-initializer list.

Q103. What is the purpose of using declaration in a derived class for a base overload set?

A. Bring hidden base overloads into derived scope

B. Make all functions virtual

C. Change private members to public

D. Delete the base class

Answer: A — Bring hidden base overloads into derived scope
Explanation: A using declaration can expose base overloads otherwise hidden by a same-named derived declaration.

Q104. What does the override specifier ensure?

A. The function is inline

B. The function is private

C. The function is called at compile time

D. The function actually overrides a virtual base function

Answer: D — The function actually overrides a virtual base function
Explanation: override makes the compiler diagnose a function that does not override a virtual base member.

Q105. What does final do on a virtual function?

A. Makes it static

B. Prevents further overriding

C. Makes it pure virtual

D. Forces inlining

Answer: B — Prevents further overriding
Explanation: A final virtual function cannot be overridden in a further-derived class.

Q106. What is a friend class?

A. A class granted access to another class's private and protected members

B. A class with no constructor

C. A nested class only

D. A class that inherits publicly

Answer: A — A class granted access to another class's private and protected members
Explanation: Friendship grants access but is not inherited, reciprocal or transitive.

Q107. What is the difference between initialization and assignment?

A. Assignment only applies to pointers

B. They are always identical

C. Initialization establishes an object's initial state; assignment changes an already-existing object

D. Initialization only occurs at runtime

Answer: C — Initialization establishes an object's initial state; assignment changes an already-existing object
Explanation: Initialization occurs as an object is created; assignment operates on an existing object.

Q108. What does std::array provide?

A. A hash table

B. A fixed-size array wrapper with STL interfaces

C. A dynamically growing linked list

D. A resizable vector

Answer: B — A fixed-size array wrapper with STL interfaces
Explanation: std::array stores a fixed number of elements as part of the object.

Q109. What is the main purpose of std::span?

A. An associative container

B. A linked list

C. A non-owning view over a contiguous sequence

D. Owning dynamic storage

Answer: C — A non-owning view over a contiguous sequence
Explanation: span is a lightweight non-owning view over contiguous elements.

Q110. What is undefined behaviour in C++?

A. A compiler warning

B. A guaranteed exception

C. A behaviour fully specified by the standard

D. Behaviour for which the standard imposes no requirements

Answer: D — Behaviour for which the standard imposes no requirements
Explanation: Undefined behaviour means the language standard places no requirements on the program's behaviour.

Q111. What is an ODR violation?

A. Declaring an overloaded function

B. Using a namespace

C. Using a variable once

D. Violating the One Definition Rule

Answer: D — Violating the One Definition Rule
Explanation: The One Definition Rule governs definitions of entities across translation units and within a program.

Q112. What is the purpose of inline in modern C++ beyond optimization hints?

A. It permits equivalent definitions across translation units under ODR conditions

B. It guarantees machine-code inlining

C. It disables linkage

D. It makes a function virtual

Answer: A — It permits equivalent definitions across translation units under ODR conditions
Explanation: inline is primarily an ODR/linkage property; it does not guarantee compiler inlining.

Q113. What is a translation unit in C++?

A. A class definition

B. A running thread

C. A source file after preprocessing

D. A single expression

Answer: C — A source file after preprocessing
Explanation: A translation unit is the result of preprocessing a source file and its included content.

Q114. What is the difference between #include and import in C++ modules?

A. Import is only for C headers

B. No difference in all contexts

C. Include is runtime loading

D. Modules provide named, compiled interfaces and avoid textual inclusion model

Answer: D — Modules provide named, compiled interfaces and avoid textual inclusion model
Explanation: C++ modules introduce module interfaces and imports, unlike textual header inclusion.

Q115. What is the purpose of std::string_view?

A. A stream buffer

B. A string allocator

C. A non-owning view of a character sequence

D. Owning mutable string storage

Answer: C — A non-owning view of a character sequence
Explanation: string_view does not own the characters; the referenced storage must outlive the view.

Q116. Which issue can arise from returning a string_view to a local std::string?

A. No issue because view owns data

B. Dangling view after local string destruction

C. Automatic deep copy

D. Compile-time garbage collection

Answer: B — Dangling view after local string destruction
Explanation: string_view is non-owning; a view to a destroyed local string dangles.

Q117. What is a data race in C++?

A. Two threads read the same immutable value

B. Conflicting unsynchronized accesses from different threads, at least one a write

C. A race in an algorithm

D. Two functions have same name

Answer: B — Conflicting unsynchronized accesses from different threads, at least one a write
Explanation: A data race is undefined behaviour under the C++ memory model.

Q118. Which standard facility provides mutual exclusion?

A. std::mutex

B. std::tuple

C. std::optional

D. std::vector

Answer: A — std::mutex
Explanation: std::mutex provides mutual exclusion; RAII wrappers such as lock_guard manage locking.

Q119. What is std::lock_guard designed to do?

A. Create a new thread

B. Manually allocate a thread

C. Lock a vector

D. Acquire a mutex on construction and release it on destruction

Answer: D — Acquire a mutex on construction and release it on destruction
Explanation: lock_guard is a scope-based RAII lock wrapper.

Q120. What is the purpose of std::atomic?

A. Make all compound operations automatically transactional

B. Replace every mutex in all cases

C. Provide atomic operations on supported types

D. Make data persistent

Answer: C — Provide atomic operations on supported types
Explanation: atomic provides atomic operations and memory-order semantics for supported types; it does not make arbitrary multi-step logic atomic.

PART C — OPERATING SYSTEMS

20 questions • Attempt first, then review the explanation.

Q121. Which statement best distinguishes a process from a thread?

A. Threads always have separate address spaces

B. Processes cannot have threads

C. A thread is stored permanently on disk

D. A process is a program in execution with its own address space; threads share process resources

Answer: D — A process is a program in execution with its own address space; threads share process resources
Explanation: Threads in one process generally share code and data/address space, while each thread has its own execution state and stack.

Q122. Which scheduling algorithm can cause the convoy effect?

A. Priority with aging

B. FCFS

C. Shortest Remaining Time First

D. Round Robin

Answer: B — FCFS
Explanation: In FCFS, a long CPU-bound job can delay many short jobs behind it.

Q123. What is a context switch?

A. Changing source code

B. Changing a process's priority only

C. Moving a process to disk permanently

D. Saving/restoring execution state to switch CPU execution between tasks

Answer: D — Saving/restoring execution state to switch CPU execution between tasks
Explanation: The OS saves one task's state and restores another's so execution can switch.

Q124. Which is NOT one of the four necessary conditions for deadlock?

A. Mutual exclusion

B. Hold and wait

C. Circular wait

D. Preemption

Answer: D — Preemption
Explanation: The four conditions are mutual exclusion, hold-and-wait, no preemption and circular wait.

Q125. What does a semaphore primarily provide?

A. CPU instruction decoding

B. Virtual memory translation

C. Synchronization through a counter and atomic operations

D. A file format

Answer: C — Synchronization through a counter and atomic operations
Explanation: Semaphores coordinate access to resources and ordering between concurrent activities.

Q126. What is a mutex?

A. A CPU register

B. A mutual-exclusion lock

C. A page replacement algorithm

D. A type of process

Answer: B — A mutual-exclusion lock
Explanation: A mutex protects a critical section so only one thread owns the lock at a time.

Q127. What is a race condition?

A. A program that runs quickly

B. Outcome depends on timing/interleaving of concurrent operations

C. A deadlock by definition

D. A memory allocation policy

Answer: B — Outcome depends on timing/interleaving of concurrent operations
Explanation: A race condition occurs when correctness depends on uncontrolled execution order.

Q128. What is a critical section?

A. A section of code accessing shared state that needs synchronization

B. The boot sector

C. A page table entry

D. A compiler optimization

Answer: A — A section of code accessing shared state that needs synchronization
Explanation: Critical sections must be protected to prevent unsafe concurrent access.

Q129. What is paging?

A. Dividing virtual memory into fixed-size pages mapped to frames

B. Scheduling threads

C. Compressing files

D. Dividing memory into variable-size segments only

Answer: A — Dividing virtual memory into fixed-size pages mapped to frames
Explanation: Paging maps fixed-size virtual pages to physical frames.

Q130. What is a page fault?

A. An exception when a memory access requires OS handling because the page is not currently mapped/present as needed

B. A CPU overheating event

C. A disk formatting error

D. Always a fatal crash

Answer: A — An exception when a memory access requires OS handling because the page is not currently mapped/present as needed
Explanation: A page fault may be handled by loading a page, creating a mapping or reporting an invalid access.

Q131. What is a TLB?

A. Temporary Link Block

B. Translation Lookaside Buffer caching address translations

C. Thread Lock Buffer

D. Task Loading Bus

Answer: B — Translation Lookaside Buffer caching address translations
Explanation: The TLB caches recent virtual-to-physical address translations.

Q132. What is virtual memory?

A. An abstraction giving processes virtual address spaces backed by physical memory and storage mechanisms

B. Only RAM

C. A compiler feature

D. A CPU cache

Answer: A — An abstraction giving processes virtual address spaces backed by physical memory and storage mechanisms
Explanation: Virtual memory provides address-space abstraction, protection and mechanisms such as paging.

Q133. What is internal fragmentation?

A. Free space between files

B. Unused space inside an allocated block

C. A deadlock

D. A page miss

Answer: B — Unused space inside an allocated block
Explanation: Internal fragmentation is wasted space within allocated memory units.

Q134. What is external fragmentation?

A. Memory used by cache

B. Wasted bytes inside a fixed block

C. Free memory split into non-contiguous holes

D. A process stack overflow

Answer: C — Free memory split into non-contiguous holes
Explanation: External fragmentation occurs when total free memory exists but is split into unusable separated blocks.

Q135. Which state commonly follows a process waiting for I/O completion?

A. Blocked/waiting

B. Ready

C. Running

D. Terminated

Answer: A — Blocked/waiting
Explanation: A process waiting for an event or I/O is blocked until the event completes.

Q136. What is starvation in scheduling?

A. A process waits indefinitely because other processes are continually favoured

B. A process gets too much CPU

C. All processes terminate

D. The CPU is idle

Answer: A — A process waits indefinitely because other processes are continually favoured
Explanation: Starvation is indefinite postponement; aging can help prevent it.

Q137. What is aging in priority scheduling?

A. Deleting old processes

B. Reducing clock speed

C. Gradually increasing the priority of waiting processes

D. Reducing a process's memory

Answer: C — Gradually increasing the priority of waiting processes
Explanation: Aging raises waiting processes' priority over time to reduce starvation.

Q138. What is a system call?

A. A controlled interface through which a program requests OS services

B. A CPU cache miss

C. A compiler directive

D. A call from one user function to another

Answer: A — A controlled interface through which a program requests OS services
Explanation: System calls provide access to kernel services such as files, processes and memory.

Q139. Which memory region typically stores dynamically allocated objects?

A. Text segment

B. Instruction cache

C. Heap

D. Register file

Answer: C — Heap
Explanation: Dynamic allocation is commonly served from a heap-managed region.

Q140. Which memory region commonly stores function call frames and automatic local variables?

A. ROM

B. Stack

C. File system

D. GPU memory only

Answer: B — Stack
Explanation: Call frames and many automatic locals are commonly stored on a thread's stack, though implementation details vary.

PART D — DATABASE MANAGEMENT SYSTEMS

15 questions • Attempt first, then review the explanation.

Q141. What is a primary key?

A. Any indexed column

B. A candidate key chosen to uniquely identify rows and disallow NULL

C. A key that may contain duplicate values

D. A foreign key in another table

Answer: B — A candidate key chosen to uniquely identify rows and disallow NULL
Explanation: A primary key uniquely identifies each row and cannot contain NULL.

Q142. What is a foreign key?

A. A clustered index

B. A password

C. A constraint referencing a candidate/unique key in another or the same table

D. A key from another database only

Answer: C — A constraint referencing a candidate/unique key in another or the same table
Explanation: A foreign key enforces referential integrity between related tables.

Q143. Which normal form removes partial dependency on a candidate key?

A. 2NF

B. 3NF

C. BCNF only

D. 1NF

Answer: A — 2NF
Explanation: Second normal form requires 1NF and no non-prime attribute to depend on a proper subset of a candidate key.

Q144. Which normal form addresses transitive dependency of non-key attributes on keys?

A. 3NF

B. 5NF

C. 2NF

D. 1NF

Answer: A — 3NF
Explanation: Third normal form eliminates specified transitive dependencies, under its formal definition.

Q145. What does ACID stand for?

A. Access, Consistency, Isolation, Data

B. Atomicity, Concurrency, Integrity, Dependency

C. Accuracy, Concurrency, Indexing, Distribution

D. Atomicity, Consistency, Isolation, Durability

Answer: D — Atomicity, Consistency, Isolation, Durability
Explanation: ACID describes core transaction properties.

Q146. What does atomicity mean in transactions?

A. Transactions run in parallel

B. Data is always encrypted

C. Transactions never fail

D. All operations in a transaction happen or none do

Answer: D — All operations in a transaction happen or none do
Explanation: Atomicity makes a transaction indivisible from the application's perspective.

Q147. What does isolation aim to provide?

A. Transactions cannot be rolled back

B. Every query uses an index

C. Concurrent transactions behave according to the selected isolation level

D. Tables cannot be joined

Answer: C — Concurrent transactions behave according to the selected isolation level
Explanation: Isolation controls how concurrent transaction effects are visible to each other.

Q148. Which SQL clause filters groups after aggregation?

A. WHERE

B. ORDER BY

C. FROM

D. HAVING

Answer: D — HAVING
Explanation: HAVING filters groups after GROUP BY; WHERE filters rows before grouping.

Q149. Which SQL command removes all rows but keeps table structure?

A. TRUNCATE TABLE

B. ALTER TABLE

C. DROP TABLE

D. RENAME TABLE

Answer: A — TRUNCATE TABLE
Explanation: TRUNCATE removes table rows while retaining the table definition; transactional details vary by DBMS.

Q150. What is an index primarily used for?

A. Guaranteeing no duplicates

B. Replacing all constraints

C. Encrypting rows

D. Speeding up data retrieval at storage and maintenance cost

Answer: D — Speeding up data retrieval at storage and maintenance cost
Explanation: Indexes can improve lookups but consume storage and add write/update overhead.

Q151. What is a clustered index in systems that support the term?

A. An index that stores only NULL values

B. An index whose leaf organization determines or closely corresponds to row storage order

C. A hash function

D. An index on every column

Answer: B — An index whose leaf organization determines or closely corresponds to row storage order
Explanation: The exact implementation varies by DBMS; clustered organization affects how table rows are stored/accessed.

Q152. What is a database view?

A. A backup file

B. A stored query presented as a virtual table in the common case

C. Always a physical copy of a table

D. A transaction log

Answer: B — A stored query presented as a virtual table in the common case
Explanation: A standard view stores a query definition; materialized views store results.

Q153. What is a dirty read?

A. Reading data written by another transaction that has not committed

B. Reading committed data only

C. Reading from an index

D. Reading a deleted table

Answer: A — Reading data written by another transaction that has not committed
Explanation: A dirty read can expose changes that may later be rolled back.

Q154. What is a deadlock in a database?

A. A table with no primary key

B. A query with no WHERE clause

C. A missing index

D. Transactions wait cyclically for resources held by one another

Answer: D — Transactions wait cyclically for resources held by one another
Explanation: Database deadlocks involve cyclic waits; systems typically detect and abort one transaction.

Q155. What is the difference between UNION and UNION ALL?

A. UNION ALL removes duplicates

B. UNION removes duplicates; UNION ALL preserves them

C. They always produce different column counts

D. UNION sorts by primary key

Answer: B — UNION removes duplicates; UNION ALL preserves them
Explanation: UNION performs duplicate elimination; UNION ALL concatenates results without that step.

PART E — COMPUTER NETWORKS

15 questions • Attempt first, then review the explanation.

Q156. Which OSI layer is responsible for routing packets between networks?

A. Session

B. Network

C. Transport

D. Data Link

Answer: B — Network
Explanation: The network layer handles logical addressing and routing.

Q157. Which OSI layer provides end-to-end transport services?

A. Physical

B. Transport

C. Presentation

D. Data Link

Answer: B — Transport
Explanation: The transport layer provides end-to-end communication services such as reliability and multiplexing.

Q158. Which protocol is connection-oriented and provides reliable ordered byte-stream delivery?

A. ICMP

B. ARP

C. TCP

D. UDP

Answer: C — TCP
Explanation: TCP establishes a connection and provides reliable, ordered byte-stream delivery.

Q159. Which statement about UDP is correct?

A. It performs a three-way handshake

B. It retransmits all lost datagrams automatically

C. It guarantees delivery

D. It is connectionless and does not guarantee delivery or ordering

Answer: D — It is connectionless and does not guarantee delivery or ordering
Explanation: UDP is a connectionless datagram protocol without TCP-style reliability guarantees.

Q160. What is the purpose of DNS?

A. Routing packets between routers

B. Encrypting web traffic

C. Assigning MAC addresses

D. Resolving domain names to resource records such as IP addresses

Answer: D — Resolving domain names to resource records such as IP addresses
Explanation: DNS maps names to records including A/AAAA address records.

Q161. What does DHCP commonly provide to a host?

A. IP configuration such as address, gateway and DNS server

B. An encryption key for HTTPS

C. A MAC address permanently

D. A domain certificate

Answer: A — IP configuration such as address, gateway and DNS server
Explanation: DHCP dynamically supplies network configuration.

Q162. What is the purpose of ARP in IPv4 local networks?

A. Map an IPv4 address to a link-layer address

B. Encrypt TCP packets

C. Assign port numbers

D. Map a URL to an IP address

Answer: A — Map an IPv4 address to a link-layer address
Explanation: ARP resolves IPv4 addresses to link-layer addresses on a local network.

Q163. What is the role of a default gateway?

A. Translate domain names

B. Provide local file storage

C. Assign process IDs

D. Forward traffic destined for networks outside the local subnet

Answer: D — Forward traffic destined for networks outside the local subnet
Explanation: Hosts send off-subnet traffic to their configured router/default gateway.

Q164. Which port is the default for HTTPS?

A. 80

B. 25

C. 443

D. 21

Answer: C — 443
Explanation: HTTPS commonly uses TCP port 443; HTTP commonly uses port 80.

Q165. What is the main purpose of TLS?

A. Routing packets

B. Compressing HTML

C. Assigning IP addresses

D. Providing cryptographic security for communications, including authentication and confidentiality

Answer: D — Providing cryptographic security for communications, including authentication and confidentiality
Explanation: TLS protects data in transit and can authenticate the server, with optional client authentication.

Q166. What does a subnet mask determine?

A. The application protocol

B. The DNS server name

C. Which IP address bits identify network versus host portion

D. The TCP window size

Answer: C — Which IP address bits identify network versus host portion
Explanation: A subnet mask/prefix identifies the network portion of an IPv4 address.

Q167. What is the difference between a MAC address and an IP address?

A. MAC is a link-layer address; IP is a network-layer logical address

B. MAC is used only on the internet

C. IP addresses never change

D. They are always identical

Answer: A — MAC is a link-layer address; IP is a network-layer logical address
Explanation: They operate at different networking layers and have different addressing roles.

Q168. What is the TCP three-way handshake?

A. ARP, DNS, DHCP

B. SYN, SYN-ACK, ACK

C. ACK, FIN, RST

D. GET, POST, PUT

Answer: B — SYN, SYN-ACK, ACK
Explanation: TCP commonly establishes a connection using SYN, SYN-ACK and ACK.

Q169. What is a socket?

A. A routing protocol

B. A type of encryption

C. An endpoint abstraction for network communication, commonly identified by address and port

D. A physical network cable

Answer: C — An endpoint abstraction for network communication, commonly identified by address and port
Explanation: Sockets provide an API abstraction for sending and receiving data over networks.

Q170. What is packet loss?

A. Packets failing to reach their destination

B. A packet arriving early

C. A DNS record update

D. A change in IP address

Answer: A — Packets failing to reach their destination
Explanation: Packet loss occurs when transmitted packets do not arrive at the destination.

PART F — EMBEDDED SYSTEMS & COMPUTER FUNDAMENTALS

10 questions • Attempt first, then review the explanation.

Q171. What is an embedded system?

A. A programming language

B. A database server

C. A general-purpose desktop only

D. A computer system designed for a specific function within a larger product

Answer: D — A computer system designed for a specific function within a larger product
Explanation: Embedded systems combine hardware and software to perform dedicated functions.

Q172. What is a microcontroller?

A. A network router only

B. A type of compiler

C. A CPU without memory or peripherals

D. A chip integrating CPU, memory and peripherals for control applications

Answer: D — A chip integrating CPU, memory and peripherals for control applications
Explanation: Microcontrollers integrate processing, memory and peripherals on one chip.

Q173. What is the difference between a microprocessor and microcontroller?

A. A microcontroller cannot run code

B. No distinction exists

C. A microprocessor always consumes less power

D. A microprocessor typically relies on external memory/peripherals; a microcontroller integrates more system components

Answer: D — A microprocessor typically relies on external memory/peripherals; a microcontroller integrates more system components
Explanation: This is a general architectural distinction; modern products vary.

Q174. What is firmware?

A. Software stored in non-volatile memory that controls hardware/device behaviour

B. A mechanical component

C. A network cable

D. A cloud database

Answer: A — Software stored in non-volatile memory that controls hardware/device behaviour
Explanation: Firmware is low-level software closely associated with device hardware.

Q175. What is an interrupt?

A. An event that causes the processor to suspend normal flow and execute an interrupt handler

B. A compiler warning

C. A memory leak

D. A permanent shutdown

Answer: A — An event that causes the processor to suspend normal flow and execute an interrupt handler
Explanation: Interrupts allow hardware/software events to request timely processor attention.

Q176. Why should an interrupt service routine (ISR) usually be short?

A. To prevent compilation

B. To reduce code readability

C. To minimize interrupt latency and avoid delaying other work

D. To increase heap usage

Answer: C — To minimize interrupt latency and avoid delaying other work
Explanation: Short ISRs improve responsiveness; deferred processing can handle longer tasks.

Q177. What does memory-mapped I/O mean?

A. All I/O uses files

B. The CPU cannot access peripherals

C. Device registers are accessed through addresses in the memory address space

D. I/O is performed only by DMA

Answer: C — Device registers are accessed through addresses in the memory address space
Explanation: Memory-mapped I/O exposes peripheral registers through addressable locations.

Q178. What is polling?

A. A hardware reset

B. A memory allocation method

C. A routing algorithm

D. Waiting for an event by repeatedly checking a status condition

Answer: D — Waiting for an event by repeatedly checking a status condition
Explanation: Polling repeatedly checks device or state status, potentially wasting CPU cycles.

Q179. What is DMA?

A. Data Mapping Algorithm

B. Digital Memory Array

C. Dynamic Module Allocation

D. Direct Memory Access

Answer: D — Direct Memory Access
Explanation: DMA allows devices/controllers to transfer data to/from memory with limited CPU involvement.

Q180. What is a watchdog timer?

A. A hardware timer that can reset/recover a system if software fails to service it

B. A network protocol

C. A timer that displays time

D. A clock for scheduling meetings

Answer: A — A hardware timer that can reset/recover a system if software fails to service it
Explanation: A watchdog detects stalled or faulty execution when not serviced within a configured interval.

PART G — SOFTWARE ENGINEERING & TESTING

10 questions • Attempt first, then review the explanation.

Q181. What is unit testing?

A. Testing network bandwidth

B. Testing user acceptance only

C. Testing the entire deployed system only

D. Testing a small unit or component in isolation

Answer: D — Testing a small unit or component in isolation
Explanation: Unit tests focus on individual functions/classes/components, often with dependencies isolated.

Q182. What is integration testing?

A. Testing interactions between combined modules or systems

B. Testing only a single statement

C. Testing only UI colours

D. Testing production load exclusively

Answer: A — Testing interactions between combined modules or systems
Explanation: Integration tests reveal issues at module boundaries and interfaces.

Q183. What is regression testing?

A. Testing hardware voltage

B. Testing without expected results

C. Testing only new features

D. Re-running tests to ensure changes have not broken existing behaviour

Answer: D — Re-running tests to ensure changes have not broken existing behaviour
Explanation: Regression testing checks that existing functionality remains correct after changes.

Q184. What is a compiler?

A. A program that translates source code into another form, often machine/object code

B. A CPU scheduler

C. A database index

D. A network switch

Answer: A — A program that translates source code into another form, often machine/object code
Explanation: A compiler translates source programs into target representations; toolchains may include several stages.

Q185. What is the role of a linker?

A. Execute source code line by line

B. Allocate runtime stack frames only

C. Combine object files/libraries and resolve symbols into an executable or library

D. Format source code

Answer: C — Combine object files/libraries and resolve symbols into an executable or library
Explanation: The linker resolves external references and combines compiled units and libraries.

Q186. What is debugging?

A. Only compiling code

B. Adding features without testing

C. Deleting logs

D. Systematically locating, understanding and correcting defects

Answer: D — Systematically locating, understanding and correcting defects
Explanation: Debugging uses reproduction, inspection, tracing and tests to isolate defects.

Q187. What is a race condition in software?

A. A failed database backup

B. A result that depends on timing/interleaving of concurrent operations

C. A syntax error

D. A slow algorithm

Answer: B — A result that depends on timing/interleaving of concurrent operations
Explanation: Race conditions occur when unsynchronized concurrent actions affect correctness.

Q188. What is a code review?

A. A performance benchmark only

B. A compiler optimization

C. A systematic peer examination of source code for defects, maintainability and standards

D. A production deployment

Answer: C — A systematic peer examination of source code for defects, maintainability and standards
Explanation: Code reviews improve quality by identifying defects and design issues before or after merging.

Q189. What does API stand for?

A. Advanced Process Integration

B. Application Protocol Internet

C. Application Programming Interface

D. Automated Program Instruction

Answer: C — Application Programming Interface
Explanation: An API defines how software components interact.

Q190. What is version control used for?

A. Tracking and managing changes to files and collaborative code

B. Encrypting RAM

C. Replacing testing

D. Increasing CPU speed

Answer: A — Tracking and managing changes to files and collaborative code
Explanation: Version control systems record history, support branching/merging and enable collaboration.

PART H — DSA & ALGORITHM FUNDAMENTALS (MCQ CONCEPTS)

10 questions • Attempt first, then review the explanation.

Q191. What is the worst-case time complexity of binary search on a sorted array?

A. O(n log n)

B. O(1)

C. O(n)

D. O(log n)

Answer: D — O(log n)
Explanation: Each comparison halves the remaining search interval, giving logarithmic worst-case time.

Q192. Which sorting algorithm is stable in its standard implementation?

A. Quick sort

B. Selection sort

C. Heap sort

D. Merge sort

Answer: D — Merge sort
Explanation: Merge sort can be implemented stably by choosing the left element first on equal keys.

Q193. What is the worst-case time complexity of quicksort with consistently poor pivots?

A. O(log n)

B. O(n log n)

C. $O(n^2)$

D. O(n)

Answer: C — $O(n^2)$
Explanation: Highly unbalanced partitions lead to quadratic work.

Q194. Which data structure is most suitable for implementing breadth-first search?

A. Stack

B. Heap

C. Queue

D. Hash set only

Answer: C — Queue
Explanation: BFS explores vertices in increasing distance layers using a FIFO queue.

Q195. Which data structure is naturally used by depth-first search?

A. Stack

B. Priority queue

C. Queue

D. Circular buffer only

Answer: A — Stack
Explanation: DFS uses a stack explicitly or the program call stack recursively.

Q196. What is the average-case lookup complexity of a well-distributed hash table?

A. $O(n^2)$

B. O(1)

C. O(log n)

D. O(n log n)

Answer: B — O(1)
Explanation: With a suitable hash function and controlled load factor, average lookup is expected constant time.

Q197. What is the time complexity of inserting an element at the head of a singly linked list when the head pointer is available?

A. O(log n)

B. O(1)

C. O(n log n)

D. O(n)

Answer: B — O(1)
Explanation: Only the new node's next pointer and head pointer need updating.

Q198. Which traversal of a binary search tree visits keys in non-decreasing order?

A. Level order

B. Inorder

C. Postorder

D. Preorder

Answer: B — Inorder
Explanation: Inorder traversal visits left subtree, node, then right subtree.

Q199. What is the auxiliary space complexity of recursive DFS on a graph in the worst case?

A. $O(E^2)$

B. O(V)

C. O(log V) always

D. O(1)

Answer: B — O(V)
Explanation: The recursion stack and visited state can require O(V) space.

Q200. Which technique is most suitable for finding the shortest path in an unweighted graph?

A. Breadth-first search

B. Topological sort on any graph

C. Depth-first search

D. Dijkstra with negative weights

Answer: A — Breadth-first search
Explanation: BFS finds shortest paths by edge count in unweighted graphs.

Answer Distribution & Trainer Use

Options have been shuffled independently across questions to avoid a predictable answer-letter pattern. Students should explain their reasoning rather than memorize answer letters.

Suggested classroom use

- Diagnostic: select 20 mixed questions and give students 25 minutes.
- Topic teaching: use 10–15 questions immediately after teaching each topic.
- Revision: ask students to maintain an error log with the topic, mistake and corrected concept.
- Final practice: select a mixed set under timed conditions and discuss every incorrect answer.

Recommended error-log columns: Question number | Topic | My selected option | Correct concept | Why I made the mistake | Revision date.

Scope note: This is Part 1 of the larger training material. It contains MCQs only. Pseudocode and C/C++ output-tracing exercises, followed by a separate DSA question bank with coding problems and solutions, can be developed as the next parts.